

CSE 331: Computer Security Fundamentals Spring 2026

Program Analysis and
Automatic Vulnerability Discovery

Amir Rahmati
Xinrui Yu

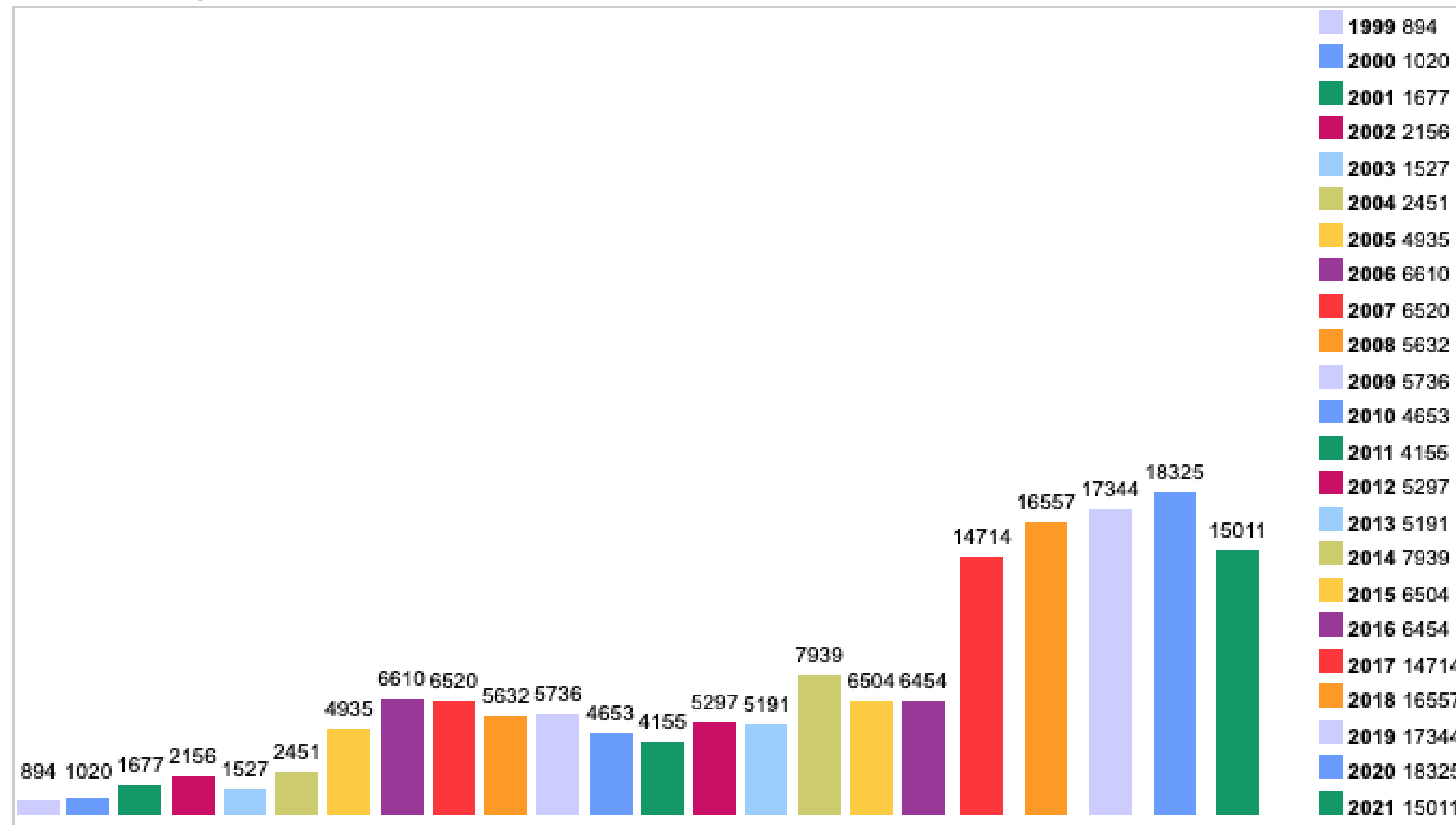


Stony Brook Institute
at Anhui University

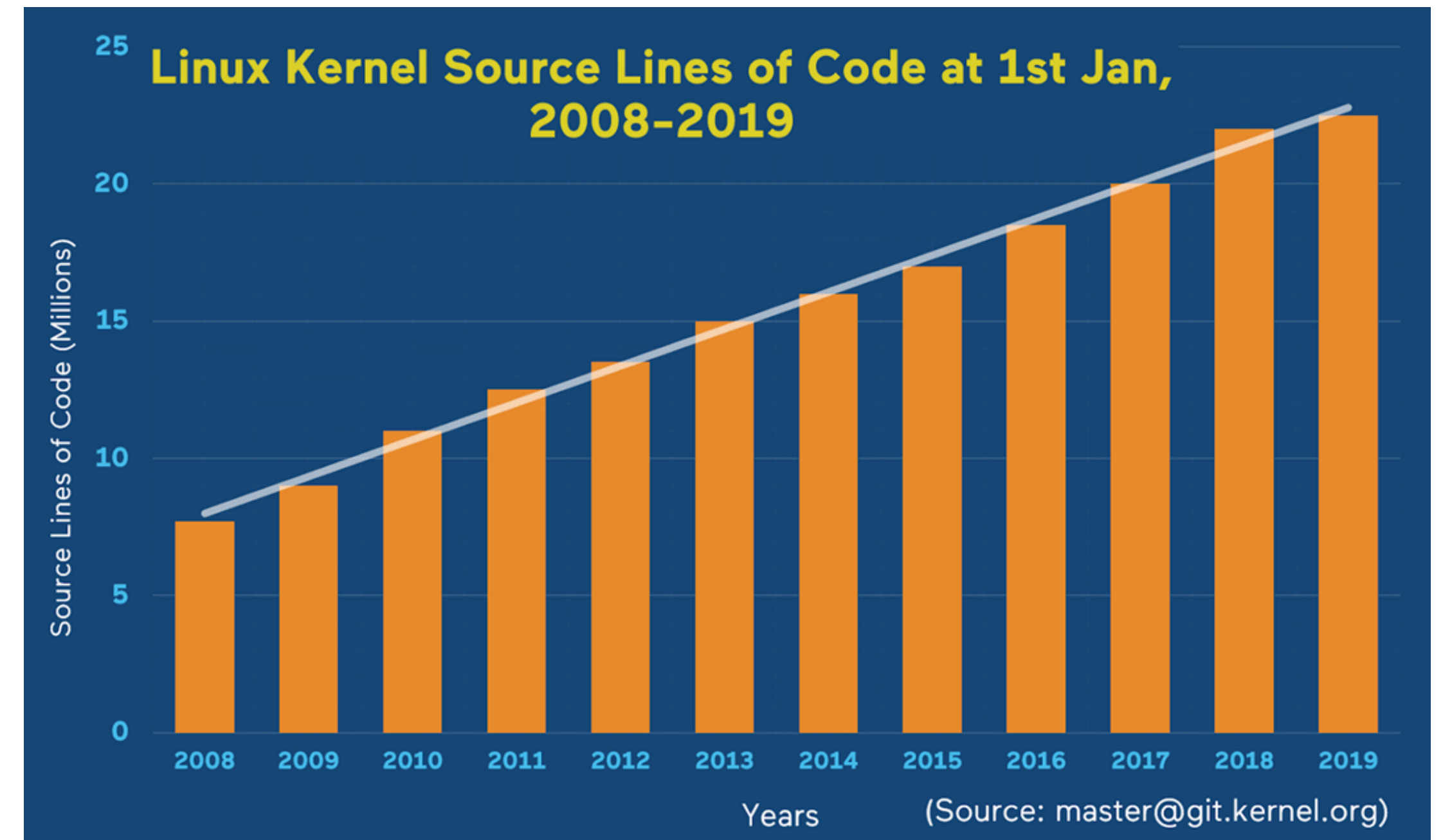
安徽大学纽约石溪学院

Why automatic discovery?

- Finding bugs (and exploits) are often hard
- We're not keeping up...



Common Vulnerabilities and Exposures



Approaches to Finding Bugs

- **Verification:**
 - Write specification —> Prove that the code follows the specification.
 - Making headways in the past decade. Still has a long way to go.
- **Testing:**
 - Check program against various inputs to show it behaves correctly.
 - Targets known bugs
- **Code Auditing**
 - Have someone else verify the code
 - Expansive and error-prone
- **Program Analysis**

Analysis Paradigms

- Static: find flaws without executing the program
- Dynamic: execute the program to find flaws.

Static Analysis

- Pros:
 - Don't need to execute the program!
 - Can make guarantees of spotting all instances of specific vulnerability classes.
 - Encourage better development practices
- Cons:
 - Typically require source code.
 - Incredible imprecise and time consuming.
 - Can only analyze limited properties
- "There is a buffer overflow on every line of the program" will technically spot all instances of buffer overflows...

What can static analysis do?

The Halting Problem

- Can we write an analyzer that can prove, for any program P and inputs to it, P will terminate.
- Provably undecidable:
- Assume there exists a program Halt(P) that solves the halting problem.

- What happens with Halt(Paradox())?

```
Paradox()  
{  
    while !Halt(Paradox());  
    Terminate;  
}
```

- If True, then while(0) → Never terminate → contradiction
- If False, then while(1) → Should terminate → contradiction

So what can Static Analysis do?

- All array accesses are in bounds?
- Is there a use after free problem in this code?
- ...
- Again, undecidable. Can be reduced from the halting problem.

All array accesses are in bounds?

- Assume there exists a program `Index(P)` that solves the problem above.
- What happens with `Index(Paradox_2())`?

```
Paradox_2()
{
    int a[3];
    if and only if (halts)
        a[10] = 0;
}
```

- If out of bounds, then `Paradox_2()` must halt;
- If not out of bounds, then `Paradox_2()` mustn't halt.
- So, you can use `Index(P)` to solve the halting problem – which is impossible. So, there is no `Index(P)`.

Rice's theorem:
All non-trivial semantic properties of
programs are undecidable.

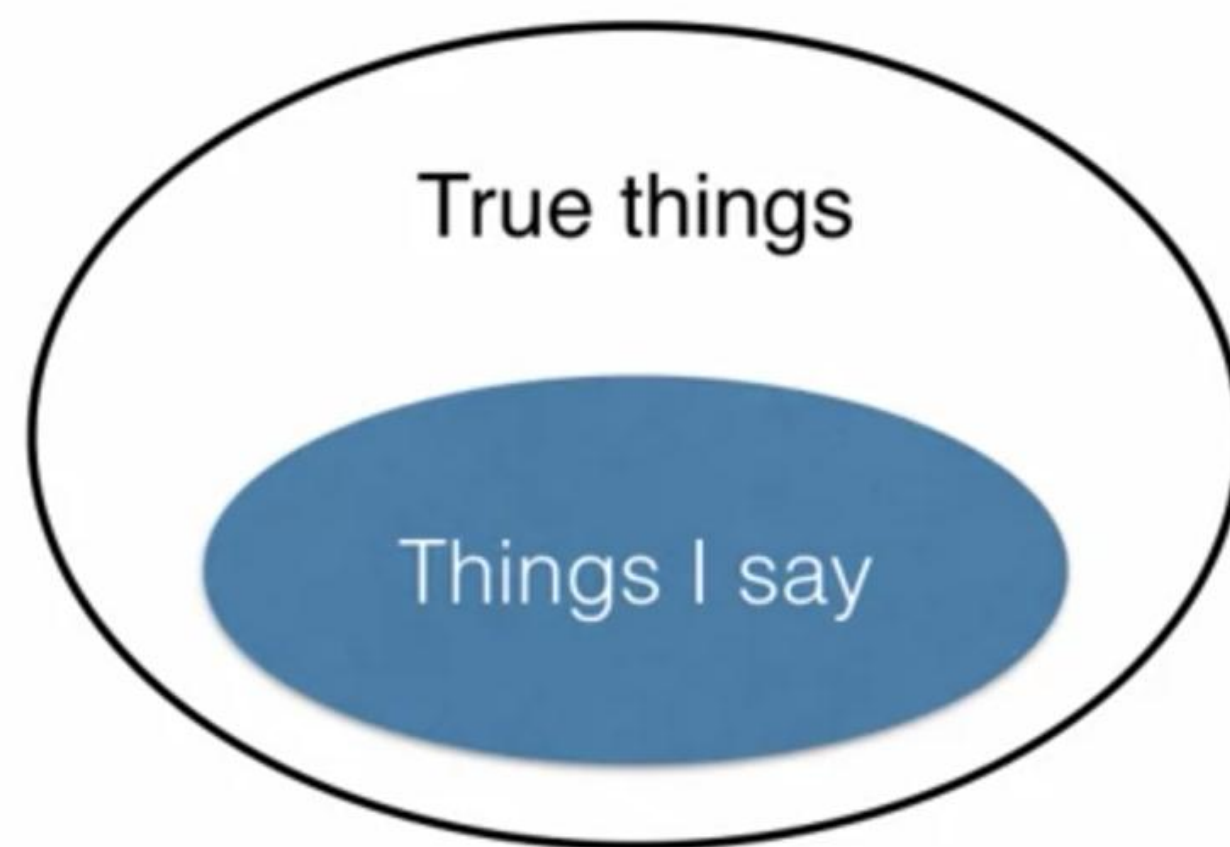
Non-trivial: neither true for every program, nor false for every program.
Semantic: relating to meaning in language or logic, as opposed to syntactic

Umm... what then?

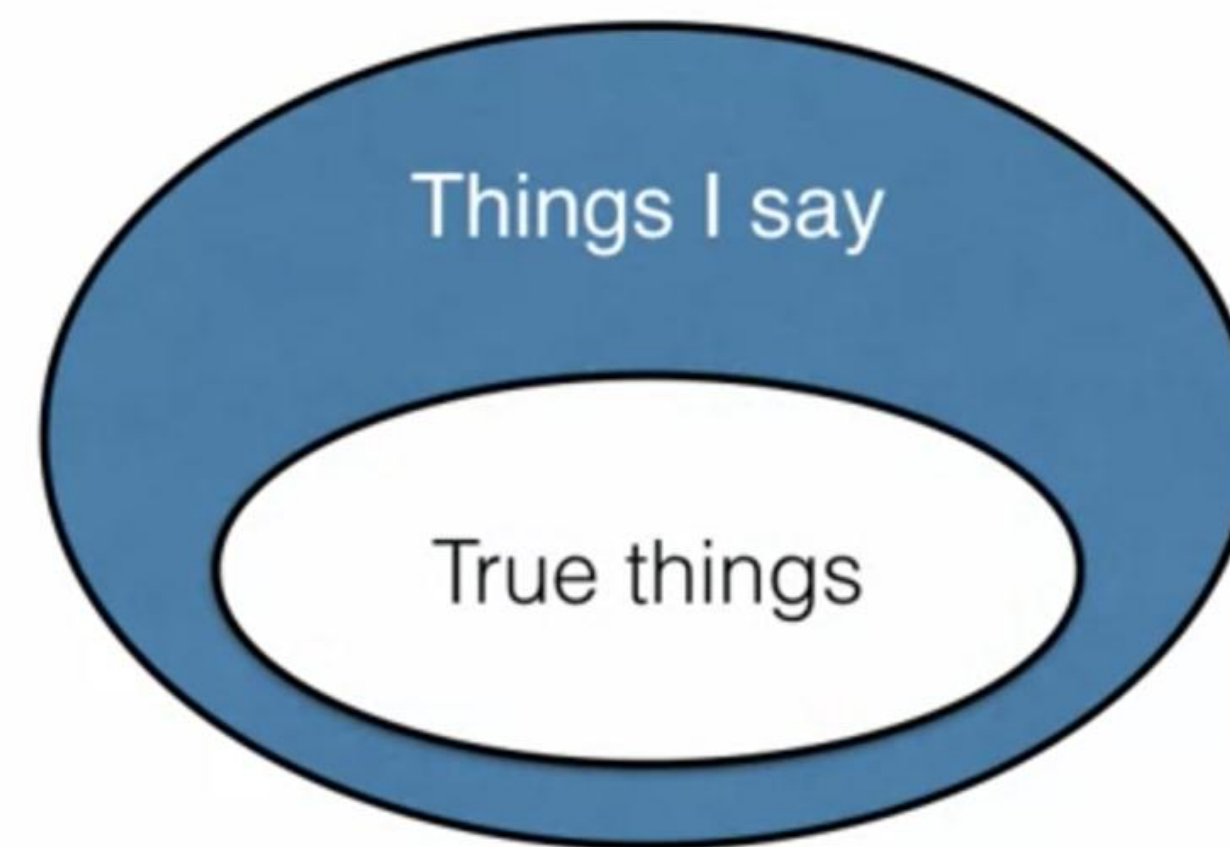
- **Perfect** static analysis is **not possible**.
- **Imperfect** static analysis is perfectly **possible**.
- Possible imperfections:
 - **Nontermination**: Analyzer never terminates
 - **False positive**: Claim error where there isn't any
 - **False negative**: Missing errors
- Soundness vs completeness tradeoff

Soundness vs Completeness

- Soundness: If the program is claimed to be error free, then it really is (**TNR = 1**)
 - Alarms do not imply problems
- Completeness: If the program is claimed to be erroneous, then it really is (**TPR = 1**)
 - Silence does not imply being error-free
- Tools are usually neither sound nor complete (but usually lean toward one)



Trivially Sound: Say nothing



Trivially Complete: Say everything

Flow Analysis

- How values may flow between different memory locations.
- Catch errors rooting from **trusting unvalidated input**.
- input from user is **tainted**.
- various operations expect to work on untainted data.
 - source string of strcpy()
 - format string of printf()

Example

Adversary-controlled format string

```
char *name = fgets(..., network_fd);  
printf(name); // Oops
```

tainted: possibly controlled by adversary

untainted: must not be controlled by adversary

```
int printf(untainted char *fmt, ...);  
tainted char *fgets(...);
```

tainted annotation: untrusted source

untainted: trusted sink

```
tainted char *name = fgets(..., network_fd);  
printf(name); // FAIL: tainted ≠ untainted
```

Need to infer which sources can reach which sinks

Legal vs. Illegal Flows

Legal Flow

```
void f(tainted int);  
untainted int a = ...;  
f(a);
```

f accepts **tainted** or **untainted** data

untainted $\leq$ **tainted**

Allowed flow as a
lattice

Illegal Flow

```
void g(untainted int);  
tainted int b = ...;  
g(b);
```

g accepts *only* **untainted** data

tainted $\not\leq$ **untainted**

untainted $<$ **tainted**

Analysis Approach

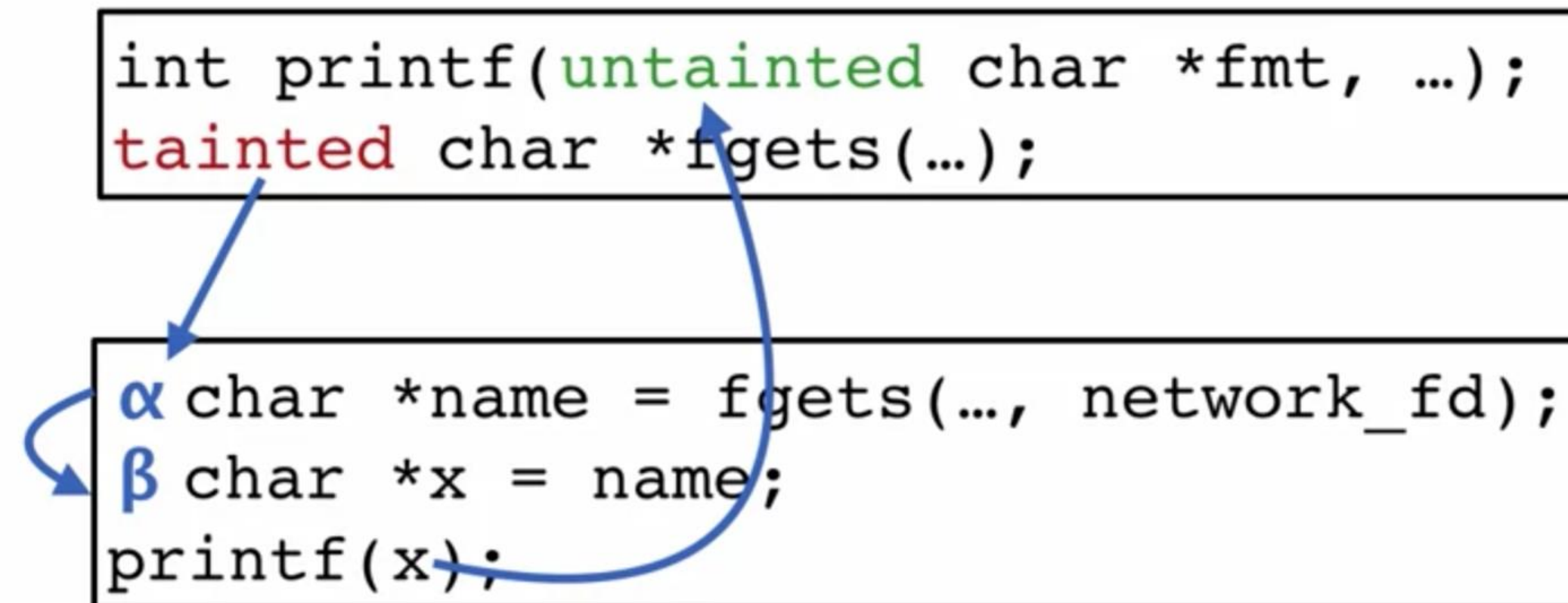
- Flow analysis $\rightarrow$ Type inference
- Steps:
- **Create a name** for each missing qualifier (α, β ..)
- For each statement in the program, **generate constraints** ($q_1 \leq q_2$)
 - E.g., $x = y \rightarrow q_y \leq q_x$
- **Solve the constraints** to produce solutions for the missing qualifiers
 - A solution is a substitution of qualifiers (like tainted or untainted) for names (like α and β) such that all of the constraints are legal flows
- No solution $\rightarrow$ Possible illegal flow

Back to our example

```
int printf(untainted char *fmt, ...);  
tainted char *fgets(...);
```

```
α char *name = fgets(..., network_fd);  
β char *x = name;  
printf(x);
```

Back to our example



tainted $\leq$ α

$\alpha \leq \beta$

$\beta \leq$ **untainted**

Constraints unsolvable

Illegal Flow

What about conditionals?

Flow Sensitivity

```
int printf(untainted char *fmt, ...);  
tainted char *fgets(...);
```

```
 $\alpha$  char *name = fgets(..., network_fd);  
 $\beta$  char *x;  
if (...) x = name;  
else x = "hello!";  
printf(x);
```

tainted $\leq \alpha$

$\alpha \leq \beta$

untainted $\leq \beta$

$\beta \leq$ **untainted**

Constraints unsolvable
Illegal Flow

```
int printf(untainted char *fmt, ...);  
tainted char *fgets(...);
```

```
 $\alpha$  char *name = fgets(..., network_fd);  
 $\beta$  char *x;  
x = name;  
x = "hello!";  
printf(x);
```

tainted $\leq \alpha$

$\alpha \leq \beta$

untainted $\leq \beta$

$\beta \leq$ **untainted**

Same Constraints
Different Semantics
False Alarm

Flow Sensitivity

- What is the problem?
 - Flow insensitivity!
- Solution:
 - Assign to a variable at most once
 - Static Single Assignment (SSA)

```
int printf(untainted char *fmt, ...);  
tainted char *fgets(...);
```

```
 $\alpha$  char *name = fgets(..., network_fd);  
 $\beta$  char *x1,  $\gamma$  *x2;  
x1 = name;  
x2 = "%s";  
printf(x2);
```

tainted $\leq \alpha$

$\alpha \leq \beta$

untainted $\leq \gamma$

$\gamma \leq$ **untainted**

No Alarm

Good solution exists:

$\gamma =$ **untainted**

$\alpha = \beta =$ **tainted**

Multiple Conditionals

```
int printf(untainted char *fmt, ...);  
tainted char *fgets(...);
```

```
void f(int x) {  
     $\alpha$  char *y;  
    if (x) y = "hello!";  
    else   y = fgets(..., network_fd);  
    if (x) printf(y);  
}
```

untainted $\leq$ α

tainted $\leq$ α

$\alpha \leq$ **untainted**

tainted $\leq$ $\alpha \leq$ **untainted**

no solution for α

False Alarm!

(and flow sensitivity won't help)

Path Sensitivity

- An analysis may consider *path feasibility*. E.g.,
 $f(x)$ can execute path

- **1-2-4-5-6** when x is *not* 0, or
- **1-3-4-6** when x is 0. But,
- path **1-3-4-5-6** *infeasible*

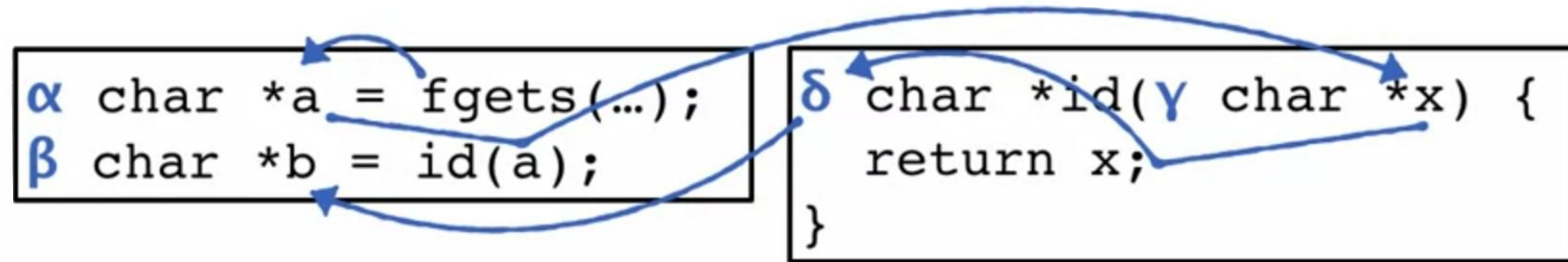
```
void f(int x) {  
    char *y;  
    1if (x) 2y = "hello!";  
    else 3y = fgets(...);  
    4if (x) 5printf(y);  
    6}
```

- A **path sensitive analysis** checks feasibility, e.g.,
by qualifying each constraint with a **path condition**
 - $x \neq 0 \Rightarrow$ **untainted** $\leq \alpha$ (segment 1-2)
 - $x = 0 \Rightarrow$ **tainted** $\leq \alpha$ (segment 1-3)
 - $x \neq 0 \Rightarrow \alpha \leq$ **untainted** (segment 4-5)

Trade off?

- Flow sensitivity adds precision, and path sensitivity adds even more, which is good
 - But both of these make solving more difficult
 - Flow sensitivity also increases the number of nodes in the constraint graph
- Path sensitivity requires more general solving procedures to handle path conditions
- Precision (often) trades off scalability
- Ultimately, limits the size of programs we can analyze

Context-Sensitive Analysis



- Names for arguments and return value
- Calls create flows
 - from caller's data to callee's arguments,
 - from callee's result to caller's returned value

tainted $\leq \alpha$

$\alpha \leq \gamma$

$\gamma \leq \delta$

$\delta \leq \beta$

Context-Sensitive Analysis

```
 $\alpha$  char *a = fgets(...);  
 $\beta$  char *b = id(a);  
 $\omega$  char *c = "hi";  
printf(c);
```

```
 $\delta$  char *id( $\gamma$  char *x) {  
    return x;  
}
```

$\text{tainted} \leq \alpha$

$\alpha \leq \gamma$

$\gamma \leq \delta$

$\delta \leq \beta$

$\text{untainted} \leq \omega$

$\omega \leq \text{untainted}$

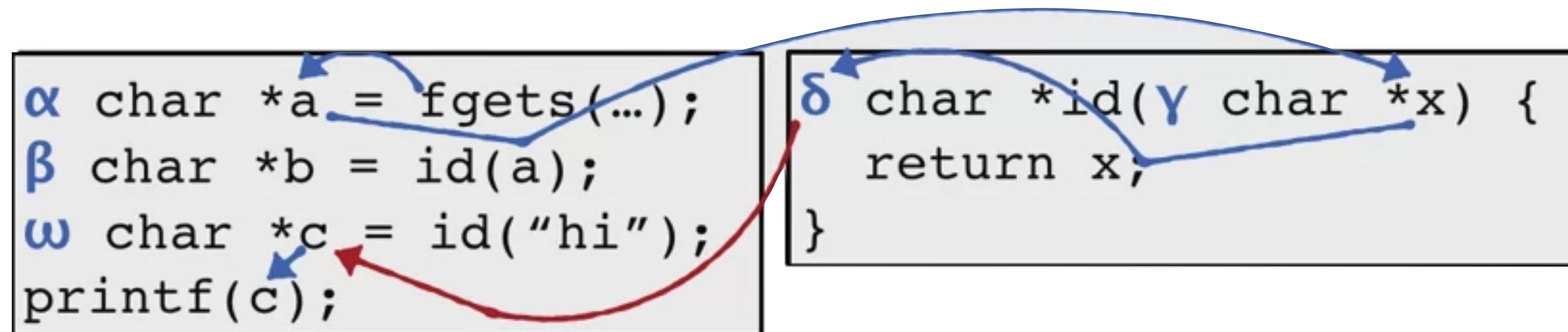
No Alarm

Good solution exists:

$\omega = \text{untainted}$

$\alpha = \beta = \gamma = \delta = \text{tainted}$

Context-Sensitive Analysis



tainted $\leq \alpha$

$\alpha \leq \gamma$

$\gamma \leq \delta$

$\delta \leq \beta$

untainted $\leq \gamma$

$\delta \leq \omega$

$\omega \leq$ **untainted**

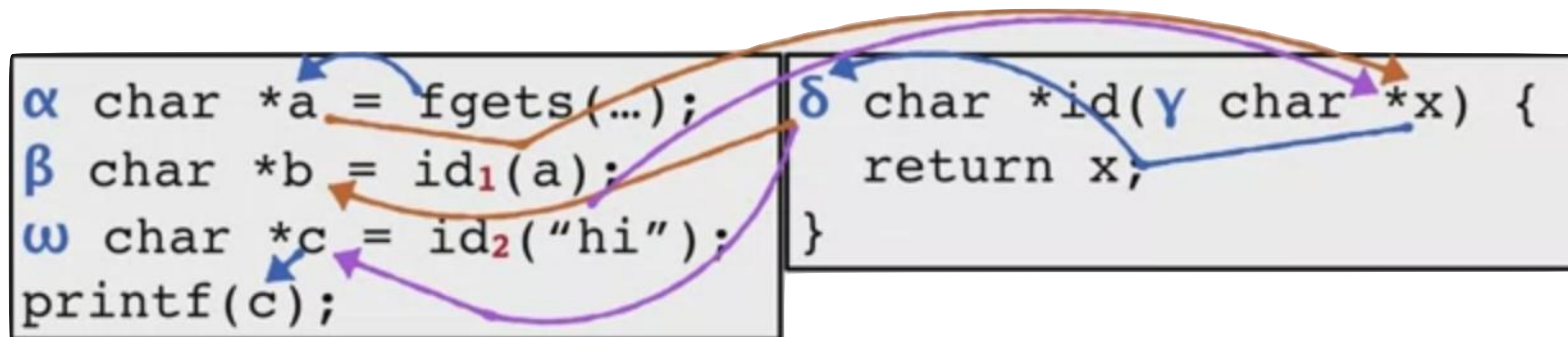
False Alarm!

No solution, and yet
no true tainted flow

Context Insensitivity

- This is a problem of **context insensitivity**
 - All call sites are “conflated” in the graph
- **Context sensitivity** solves this problem by
 - **distinguishing call sites** in some way
 - We can give them a label ***i***, e.g., the line number in the program
 - **matching up calls** with the corresponding **returns**
 - Label call and return edges
 - Allow flows if the labels and **polarities** match
 - Use index **-i** for **argument passing**, i.e., $q1 \leq -i q2$
 - Use index **+i** for **returned values**, i.e., $q1 \leq +i q2$

Context-Sensitive Analysis



tainted $\leq \alpha$

$\alpha \leq -1 \gamma$

$\gamma \leq \delta$

$\delta \leq +1 \beta$

untainted $\leq -2 \gamma$

$\delta \leq +2 \omega$

$\omega \leq \text{untainted}$

Indexes don't match
No Alarm!

Context-Sensitive Analysis

- Context sensitivity is a tradeoff again
 - Precision vs. scalability
 - $O(n)$ insensitive algorithm becomes $O(n^3)$ sensitive algorithm
 - But: sometimes higher precision improves performance
 - Eliminates infeasible paths from consideration (makes n smaller)
- Compromises possible
 - Only some call sites treated sensitively
 - Rest conflated
 - Conflate groups of call sites
 - Give them the same index
 - Sensitivity only up to a certain call depth
 - Don't do exact matching of edges beyond that depth

Pointer Analysis

```
 $\alpha$  char *a = "hi";  
( $\beta$  char *)*p = &a;  
( $\gamma$  char *)*q = p;  
 $\omega$  char *b = fgets(...);  
*q = b;  
printf(*p);
```

$\text{untainted} \leq \alpha$
 $\alpha \leq \beta$
 $\beta \leq \gamma$
 $\text{tainted} \leq \omega$
 $\omega \leq \gamma$
 $\beta \leq \text{untainted}$

Misses illegal flow!

Solution:

When assigning pointers to
pointers, add both flows

$\beta \leq \gamma$
 $\gamma \leq \beta$

Ensures aliasing constraints are sound.

Can cause false alarms.

- Reducing alarms
- If pointers are never assigned to (const) then backward flow is not needed (sound)
 - Drop backward flow edge anyway
 - Trades false alarms for missed errors (unsoundness)

Implicit Flow

```
void copy(tainted char *src,  
          untainted char *dst,  
          int len) {  
    untainted int i;  
    for (i = 0; i < len; i++) {  
        dst[i] = src[i]: //illegal  
    }  
}
```

untainted char **tainted** char

Illegal flow!

```
void copy(tainted char *src,  
          untainted char *dst,  
          int len) {  
    untainted int i, j;  
    for (i = 0; i < len; i++) {  
        for (j = 0; j < sizeof(char)*256; j++) {  
            if (src[i] == (char)j)  
                dst[i] = (char)j; //legal?  
        }  
    }  
}
```

untainted char **untainted** char

Missed Flow!

Information Flow Analysis

- The prior flow is an implicit flow, since information in one value implicitly influences another
- One way to discover these is to maintain a scoped program counter (pc) label
- Represents the maximum taint affecting the current pc
- Assignments generate constraints involving the pc
- $x = y$ produces two constraints: $\text{label}(y) \leq \text{label}(x)$ (as usual)
 $\text{pc} \leq \text{label}(x)$
- Generalized analysis tracks information flow

Information Flow Analysis

```
pc1 = untainted
pc2 = tainted
pc3 = tainted
pc4 = untainted

tainted int src;
α int dst;
if (src == 0)
    dst = 0;
else
    dst = 1;
dst += 0;
```

untainted ≤ **α**
α ≤ pc₂
untainted ≤ **α**
α ≤ pc₃
untainted ≤ **α**
α ≤ pc₄

```
tainted int src;
α int dst;
if (src > 0) dst = 0;
else         dst = 0;
```

α ≤ pc₃ = **tainted**
Solution requires **α** = **tainted**
Discovers implicit flow

- Tracking implicit flows with a pc label can lead to false alarms
- E.g., ignores values
- Extra constraints also hurt performance
- Our copying example is pathological
- We typically don't write programs like this
- Implicit flows will have little overall influence
- Typically Ignored by taint analysis
- Can be abused by untrusted code

Challenges Still remain

Taint through operations

- tainted a; untainted b; $c=a+b$ — is c tainted? (yes, probably)

Function pointers

- What function can this call go to?
- Can flow analysis to compute possible targets

Struct fields

- Track the taintedness of the whole struct, or each field?
- Taintedness for each struct instance, or shared among all of them?
- Note: objects ~ structs + function pointers

Arrays

- Keep track of taintedness of each array element, or one element representing the whole array?

Refining Taint Analysis

- Can label additional sources and sinks
 - Array bounds accesses: must have untainted index
 - Can expand taint analysis to handle sanitizers
 - Functions to convert tainted data to untainted data
-
- Other application: Leaking confidential data
 - Don't want secret sources to go to public sinks

Other types of analysis

Pointer Analysis ("points-to" analysis)

- Determine whether pointers point to the same locations
- Shares many elements of flow analysis.
- Really advanced in the last 10 years.

Data Flow Analysis

- Invented in the early 1970's. Flow sensitive, tracks "data flow facts" about variables in the program
- Liveness analysis: which variables are still alive (could be read in the future)
- Used for allocating variables to register

Abstract interpretation

- Invented in the late 1970's as a theoretical foundation for data flow analysis, and static analysis generally.
- Associated with certain analysis algorithms

Simple Static Analysis

compare.c

```
int main()
{
    char buffer[1024];
    read(0, buffer, 1);

    if (buffer[0] == 42)
    {
        puts("Pass!");
    }
    else
    {
        puts("Fail!");
    }
}
```

```
0000000000001189 <main>:
1189:    f3 0f 1e fa    endbr64
118d:    55             push    rbp
118e:    48 89 e5       mov     rbp, rsp
1191:    48 81 ec 10 04 00 00 sub     rsp, 0x410
1198:    64 48 8b 04 25 28 00 mov     rax, QWORD PTR fs:0x28
119f:    00 00
11a1:    48 89 45 f8     mov     QWORD PTR [rbp-0x8], rax
11a5:    31 c0          xor     eax, eax
11a7:    48 8d 85 f0 fb ff ff lea     rax, [rbp-0x410]
11ae:    ba 01 00 00 00  mov     edx, 0x1
11b3:    48 89 c6       mov     rsi, rax
11b6:    bf 00 00 00 00  mov     edi, 0x0
11bb:    b8 00 00 00 00  mov     eax, 0x0
11c0:    e8 cb fe ff ff  call    1090 <read@plt>
11c5:    0f b6 85 f0 fb ff ff movzx   eax, BYTE PTR [rbp-0x410]
11cc:    3c 2a          cmp     al, 0x2a
11ce:    75 0e          jne     11de <main+0x55>
11d0:    48 8d 3d 2d 0e 00 00 lea     rdi, [rip+0xe2d] # 2004 <_IO_stdin_used+0x4>
11d7:    e8 94 fe ff ff  call    1070 <puts@plt>
11dc:    eb 0c          jmp     11ea <main+0x61>
11de:    48 8d 3d 28 0e 00 00 lea     rdi, [rip+0xe28] # 200d <_IO_stdin_used+0xd>
11e5:    e8 86 fe ff ff  call    1070 <puts@plt>
11ea:    b8 00 00 00 00  mov     eax, 0x0
11ef:    48 8b 4d f8     mov     rcx, QWORD PTR [rbp-0x8]
11f3:    64 48 33 0c 25 28 00 xor     rcx, QWORD PTR fs:0x28
11fa:    00 00
11fc:    74 05          je      1203 <main+0x7a>
11fe:    e8 7d fe ff ff  call    1080 <__stack_chk_fail@plt>
1203:    c9             leave
1204:    c3             ret
```

Simple Static Analysis

- `objdump -M intel --disassemble=main compare`
- `objdump -M intel --disassemble=main compare | grep cmp`
- `objdump -M intel --disassemble=main compare | grep cmp | sed -e "s/.*0x/"`
- `objdump -M intel --disassemble=main compare | grep cmp | sed -e "s/.*0x/" | parallel echo -e '\\x{}' | .compare`

Static Analysis Tools

- **angr** is a python library providing many different binary analysis capabilities.
- **IDA**, **Ghidra**, and **Binary Ninja** (non-cloud) are all scriptable!
- Have source code? **Clang** and **GCC**!

Dynamic Analysis

Pros:

- Typically, don't require source code.
- Very precise.

Cons:

- Must run the program to analyze it. Not trivial for:
 - complex programs
 - libraries
 - embedded devices
- Can only spot bugs if the bug is triggered.

"No bugs detected" is a quite normal result of dynamic analysis of buggy programs.

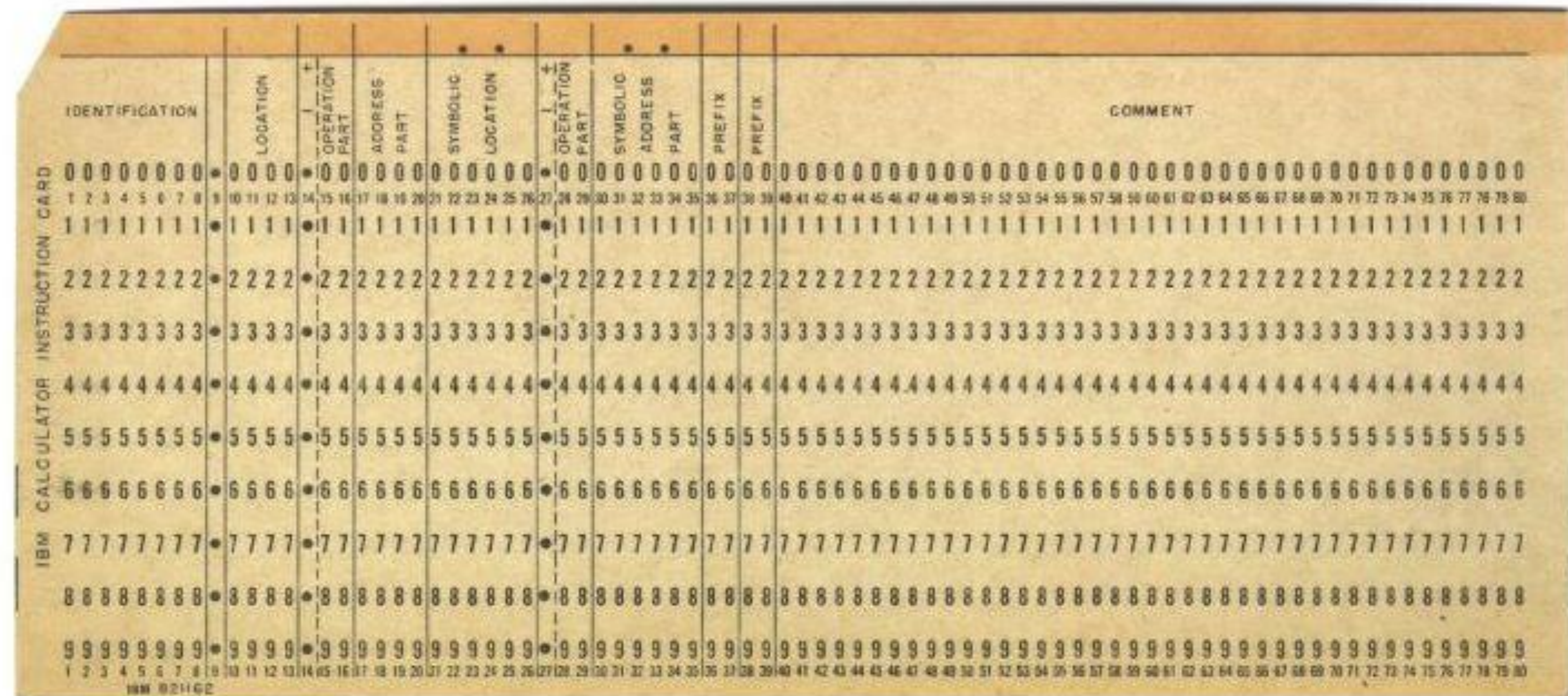
Fuzzing

1950s

- Use previously discarded punchcards as input into the program.

1981

- “A report on random testing” - Duran et al.
ACM SIGSOFT



A report on random testing

- Core concept: programmers assume that users are well-behaved.
 - Run the program.
 - Send it random data as **input**.
 - See if it crashes.

```
# while true  
> do  
>  cat /dev/urandom | monitor_for_crashes.py /program  
> done
```

- Surprisingly effective.

Monitoring for Crashes

- Fuzzers tend to find crashes. Crashes are easy to detect!
- Other types of vulnerabilities are harder, but doable:
 - Race Conditions
 - Algorithmic Attacks
 - Information Disclosures
- Lots of active research on this!

Generational Fuzzing

- Early 2010s
- Generational fuzzers use intelligent techniques to **generate** program inputs

```
# while true  
> do  
>   generate_elf_file.py /tmp/random-elf  
>   monitor_for_crashes.py objdump -d /tmp/random-elf  
> done
```

- Example:
 - Peach Fuzzer (<https://www.peach.tech/>)

Mutational Fuzzing

- Mutational fuzzers mutate existing inputs to try to trigger new program behavior (and bugs!).

```
# while true  
> do  
>  mutate_input.py /tmp/random-elf1 > /tmp/random-elf2  
>  monitor_for_crashes.py objdump -d /tmp/random-elf2  
> done
```

- Core questions:
 - How to select inputs to mutate?
 - How to mutate inputs?

Coverage-guided Mutational Fuzzing

- Modern fuzzers have the following loop:
 - Start with a provided **seed corpus** as **pending inputs**.
 - Analysis Loop
 - Execute target program with every **pending input**.
 - Record what instructions were executed when processing each **pending input**.
 - Mark inputs that increase **code coverage** as **interesting inputs**.
 - Mutate **interesting inputs** to produce a new set of **pending inputs**.
 - Find bugs!
- Some questions:
 - Seed corpus?
 - How do we identify an increase in code coverage?
 - How do we mutate inputs?

Seed Corpus

- The seed corpus is critical.
 - The **infinite monkey theorem** states that a monkey hitting keys at random on a typewriter keyboard for an infinite amount of time will almost surely type any given text, such as the complete works of William Shakespeare.
[https://en.wikipedia.org/wiki/Infinite_monkey_theorem]
- but this will take a **VERY LONG TIME**.
- Your seed corpus helps the monkeys. Make it:
 - **Varied**: it should represent a good coverage of the functionality of the program
 - **Weird**: it should trigger enough uncommon behavior to give the fuzzer a good starting point to trigger really rare behavior. Also have some normal stuff!
 - **Small**: AFL will eventually try to mutate every bit. Don't put in useless bits.

Measuring Code Coverage

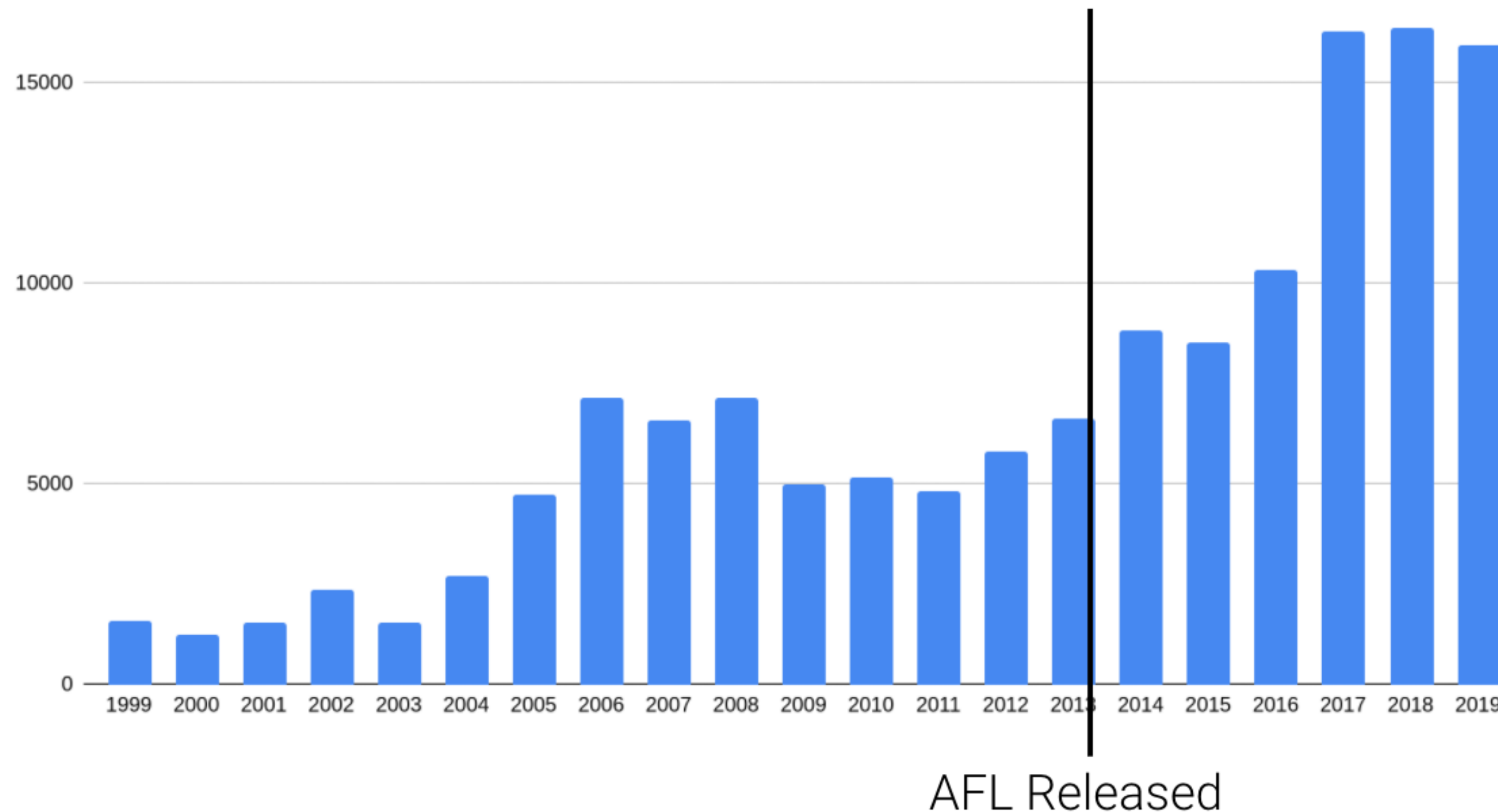
- Many coverage tracking techniques exist:
 - track each instruction that was executed
 - track each basic block that was executed
 - track each function that was executed
- American Fuzzy Lop insight: It's not the code that matters, it's how you traverse it!
- AFL measures **control flow transitions**, pairs of basic blocks connected by a jump.

Mutating Inputs

- AFL ships with several mutators:
- **bitflip L/S**: L bits toggled at a time, walking the input file with S-bit increments. Currently: 1/1, 2/1, 4/1, 8/8, 16/8, 32/8.
- **arith L/8**: subtract or add small integers to 8, 16, and 32-bit values. The stepover is 8 bits.
- **interest L/8**: list of known “interesting” 8, 16, and 32-bit values to try. The stepover is 8 bits.
- **extras**: deterministic injection (or replacement) of user-supplied or automatically determined dictionary terms.
- **havoc**: random tweaks. Operations attempted bit flips, overwrites with random and “interesting” integers, block deletion, block duplication, assorted dictionary-related operations.
- **splice**: splices together two random inputs from the queue at some arbitrarily selected midpoint.

Fuzzing is King

- Fuzzing is undeniably the best automated program analysis technique we have.
- Fuzzing finds thousands of bugs every year.



Cutting-Edge Fuzzers

- Too many fuzzers to keep track of! Notable mentions:
- **OSS-Fuzz**: continuous fuzzing for open-source software (<https://github.com/google/oss-fuzz>)
- **syzkaller**: system call fuzzing for Linux (<https://github.com/google/syzkaller>)
- **libfuzzer**: fuzzing library integrated into Clang/LLVM (<https://llvm.org/docs/LibFuzzer.html>)
- **afl++**: community-maintained AFL fork (<https://github.com/AFLplusplus/AFLplusplus>)